

**LAPORAN PRAKTIKUM  
PEMROGRAMAN MOBILE  
MODUL 3**



**BUILD A SCROLLABLE LIST**

**Oleh:**

**Akmallullail Sya'ban    NIM. 2310817310010**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
MEI 2025**

**LEMBAR PENGESAHAN**  
**LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE**  
**MODUL 3**

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Akmallullail Sya'ban  
NIM : 2310817310010

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar  
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom  
NIP. 198205082008011010

## DAFTAR ISI

|                              |    |
|------------------------------|----|
| LEMBAR PENGESAHAN.....       | 2  |
| DAFTAR ISI .....             | 3  |
| DAFTAR GAMBAR.....           | 4  |
| DAFTAR TABEL .....           | 5  |
| SOAL 1.....                  | 6  |
| A. Source Code.....          | 7  |
| B. Output Program .....      | 26 |
| C. Pembahasan .....          | 31 |
| <b>XML</b> .....             | 31 |
| <b>Jetpack Compose</b> ..... | 34 |
| D. Tautan Git.....           | 36 |
| SOAL 2.....                  | 37 |
| A. Pembahasan .....          | 37 |

## DAFTAR GAMBAR

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Gambar 1. Contoh UI List .....                                        | 6  |
| Gambar 2. Contoh UI Detail .....                                      | 7  |
| Gambar 3. Tampilan Awal UI XML .....                                  | 26 |
| Gambar 4. Tampilan Ketika di Scroll XML.....                          | 27 |
| Gambar 5. Tampilan Tombol Detail Ditekan XML .....                    | 27 |
| Gambar 6. Tampilan Awal UI di Rotate XML.....                         | 28 |
| Gambar 7. Tampilan Detail UI di Rotate XML .....                      | 28 |
| Gambar 8. Tombol IMDB di Tekan .....                                  | 28 |
| Gambar 9. Tampilan Awal UI Jetpack Compose .....                      | 29 |
| Gambar 10. Tampilan UI Ketika di Scroll Jetpack Compose .....         | 29 |
| Gambar 11. Tampilan Detail UI Jetpack Compose.....                    | 30 |
| Gambar 12. Tampilan Awal UI di Rotate Jetpack Compose .....           | 30 |
| Gambar 13. Tampilan Detail UI di Rotate Jetpack Compose .....         | 30 |
| Gambar 14. Tampilan Ketika Tombol IMDB di Tekan Jetpack Compose ..... | 31 |

## **DAFTAR TABEL**

|                                                         |    |
|---------------------------------------------------------|----|
| Tabel 1. Source Code MainActivity.kt App XML .....      | 7  |
| Tabel 2. Source Code Movie.kt App XML .....             | 8  |
| Tabel 3. Source Code ListMovieAdapter.kt App XML .....  | 8  |
| Tabel 4. Source Code HomeFragment.kt App XML .....      | 9  |
| Tabel 5. Source Code DetailFragmen.kt App XML .....     | 11 |
| Tabel 6. Source Layout XML activity_main.xml .....      | 12 |
| Tabel 7. Source Layout XML fragment_detail.xml .....    | 13 |
| Tabel 8. Source Layout XML fragment_home.xml .....      | 14 |
| Tabel 9. Source Layout XML item_movie.xml .....         | 15 |
| Tabel 10. Source String Data Jetpack Compose .....      | 17 |
| Tabel 11. Source Data Class Movie Jetpack Compose ..... | 19 |
| Tabel 12. Source List Movie Jetpack Compose .....       | 19 |
| Tabel 13. Source Detail Activity Jetpack Compose .....  | 20 |
| Tabel 14. Source Main Activity Jetpack Compose .....    | 22 |

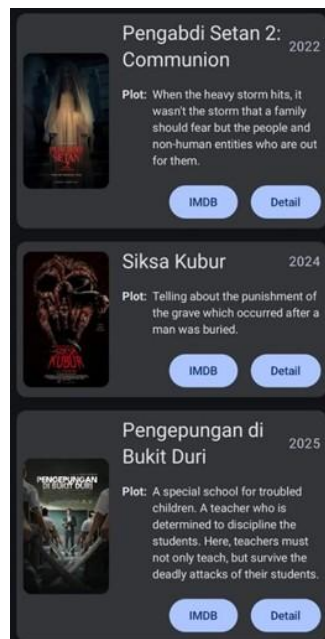
## SOAL 1

### Soal Praktikum:

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut: a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

## A. Source Code

### XML:

### MainActivity.kt

Tabel 1. Source Code MainActivity.kt App XML

|    |                                                                                     |
|----|-------------------------------------------------------------------------------------|
| 1  | <code>package com.example.movielistxml</code>                                       |
| 2  |                                                                                     |
| 3  | <code>import android.os.Bundle</code>                                               |
| 4  | <code>import android.util.Log</code>                                                |
| 5  | <code>import androidx.appcompat.app.AppCompatActivity</code>                        |
| 6  |                                                                                     |
| 7  | <code>class MainActivity : AppCompatActivity() {</code>                             |
| 8  | <code>    override fun onCreate(savedInstanceState: Bundle?) {</code>               |
| 9  | <code>        super.onCreate(savedInstanceState)</code>                             |
| 10 | <code>        setContentView(R.layout.activity_main)</code>                         |
| 11 |                                                                                     |
| 12 | <code>        val fragmentManager = supportFragmentManager</code>                   |
| 13 | <code>        val homeFragment = HomeFragment()</code>                              |
| 14 | <code>        val fragment =</code>                                                 |
|    | <code>fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)</code> |
| 15 | <code>        if (fragment != HomeFragment) {</code>                                |
| 16 | <code>            Log.d("MyFragment", "Fragment Name : " +</code>                   |
|    | <code>HomeFragment::class.java.simpleName)</code>                                   |
| 17 | <code>            fragmentManager</code>                                            |
| 18 | <code>                .beginTransaction()</code>                                    |
| 19 | <code>                .add(R.id.frame_container, homeFragment,</code>               |
|    | <code>HomeFragment::class.java.simpleName)</code>                                   |
| 20 | <code>                .commit()</code>                                              |

|    |   |
|----|---|
| 21 | } |
| 22 | } |
| 23 | } |

## Movie.kt

*Tabel 2. Source Code Movie.kt App XML*

|    |                                                 |
|----|-------------------------------------------------|
| 1  | <code>package com.example.movielistxml</code>   |
| 2  |                                                 |
| 3  | <code>import android.os.Parcelable</code>       |
| 4  | <code>import kotlinx.parcelize.Parcelize</code> |
| 5  |                                                 |
| 6  | <code>@Parcelize</code>                         |
| 7  | <code>data class Movie (</code>                 |
| 8  | <code>    val title: String,</code>             |
| 9  | <code>    val url: String,</code>               |
| 10 | <code>    val image: Int,</code>                |
| 11 | <code>    val plot: String,</code>              |
| 12 | <code>    val years: String,</code>             |
| 13 | <code>    val desc: String</code>               |
| 14 | <code>): Parcelable</code>                      |

## ListMovieAdapter.kt

*Tabel 3. Source Code ListMovieAdapter.kt App XML*

|    |                                                                                 |
|----|---------------------------------------------------------------------------------|
| 1  | <code>package com.example.movielistxml</code>                                   |
| 2  |                                                                                 |
| 3  | <code>import android.view.LayoutInflater</code>                                 |
| 4  | <code>import android.view.View</code>                                           |
| 5  | <code>import android.view.ViewGroup</code>                                      |
| 6  | <code>import android.widget.Button</code>                                       |
| 7  | <code>import android.widget.ImageView</code>                                    |
| 8  | <code>import android.widget.TextView</code>                                     |
| 9  | <code>import androidx.recyclerview.widget.RecyclerView</code>                   |
| 10 |                                                                                 |
| 11 |                                                                                 |
| 12 | <code>class ListMovieAdapter (</code>                                           |
| 13 | <code>    private val listMovie: ArrayList&lt;Movie&gt;,</code>                 |
| 14 | <code>    private val onImdbClick: (String) -&gt; Unit,</code>                  |
| 15 | <code>    private val onDetailClick: (String, Int, String, String,</code>       |
|    | <code>String) -&gt; Unit</code>                                                 |
| 16 | <code>): RecyclerView.Adapter&lt;ListMovieAdapter.ListViewHolder&gt;() {</code> |
| 17 |                                                                                 |
| 18 | <code>    class ListViewHolder(itemView: View) :</code>                         |
|    | <code>RecyclerView.ViewHolder(itemView) {</code>                                |
| 19 | <code>        val imgPoster: ImageView =</code>                                 |
|    | <code>itemView.findViewById(R.id.img_poster)</code>                             |



```

20         val tvName: TextView =
    itemView.findViewById(R.id.tv_item_title)
21         val years: TextView =
    itemView.findViewById(R.id.tv_item_years)
22         val plot: TextView =
    itemView.findViewById(R.id.tv_item_plot)
23         val desc: TextView =
    itemView.findViewById(R.id.tv_item_desc)
24         val btnImdb: Button =
    itemView.findViewById(R.id.btn_imdb)
25         val btnDetail: Button =
    itemView.findViewById(R.id.btn_detail)
26     }
27     override fun onCreateViewHolder(parent: ViewGroup,
    viewType: Int): ViewHolder{
28         val view: View =
    LayoutInflater.from(parent.context).inflate(R.layout.item_mov
    ie, parent, false)
29         return ViewHolder(view)
30     }
31     override fun getItemCount(): Int = listMovie.size
32
33     override fun onBindViewHolder(holder: ViewHolder,
    position: Int){
34         val (title, url, image, plot, years, desc) =
    listMovie[position]
35         holder.tvName.text = title
36         holder.plot.text = plot
37         holder.imgPoster.setImageResource(image)
38         holder.years.text = years
39         holder.desc.text = desc
40         holder.btnImdb.setOnClickListener{onImdbClick(url)}
41         holder.btnDetail.setOnClickListener{onDetailClick(title,
    image, plot, years, desc)}
42     }
43 }

```

## HomeFragment.kt

Tabel 4. Source Code HomeFragment.kt App XML

```

1 package com.example.movielistxml
2
3 import android.content.Intent
4 import android.net.Uri
5 import android.os.Bundle
6 import androidx.fragment.app.Fragment
7 import android.view.LayoutInflater
8 import android.view.View
9 import android.view.ViewGroup

```



```

56         )
57         binding.rvMovie.apply {
58             layoutManager =LinearLayoutManager(context)
59             adapter = movieAdapter
60             setHasFixedSize(true)
61         }
62     }
63
64     private fun getListMovie(): ArrayList<Movie>{
65         val dataMovie =
66 resources.getStringArray(R.array.data_title)
67         val dataLink =
68 resources.getStringArray(R.array.data_link)
69         val dataImage =
70 resources.obtainTypedArray(R.array.data_image)
71         val dataPlot =
72 resources.getStringArray(R.array.plot)
73         val dataYear =
74 resources.getStringArray(R.array.data_years)
75         val dataDesc =
76 resources.getStringArray(R.array.data_description)
77         val listMovie = ArrayList<Movie>()
78         for (i in dataMovie.indices){
79             val movie = Movie(dataMovie[i], dataLink[i],
80 dataImage.getResourceId(i, -1), dataPlot[i], dataYear[i],
81 dataDesc[i])
82             listMovie.add(movie)
83         }
84         dataImage.recycle()
85         return listMovie
86     }
87
88     override fun onDestroyView() {
89         super.onDestroyView()
90         _binding = null
91     }
92 }

```

## DetailFragment.kt

Tabel 5. Source Code DetailFragmen.kt App XML

```

1 package com.example.movieslistxml
2
3 import android.os.Bundle
4 import androidx.fragment.app.Fragment
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import
9 com.example.movieslistxml.databinding.FragmentDetailBinding

```

```

9
10
11 class DetailFragment : Fragment() {
12
13     private var _binding: FragmentDetailBinding? = null
14     private val binding get() = _binding!!
15
16     override fun onCreateView(
17         inflater: LayoutInflater, container: ViewGroup?,
18         savedInstanceState: Bundle?
19     ): View? {
20         _binding = FragmentDetailBinding.inflate(inflater,
21             container, false)
22
23         val title = arguments?.getString("EXTRA_NAME")
24         val image = arguments?.getInt("EXTRA_PHOTO")
25         val years = arguments?.getString("EXTRA_YEARS")
26         val plot = arguments?.getString("EXTRA_PLOT")
27         val desc = arguments?.getString("EXTRA_DESC")
28
29         binding.tvItemTitle.text = title
30         binding.tvItemPlot.text = years
31         binding.tvItemYears.text = plot
32         binding.tvItemDesc.text = desc
33         image?.let {
34             binding.imgPoster.setImageResource(it)
35         }
36         return binding.root
37     }
38
39     override fun onDestroyView() {
40         super.onDestroyView()
41         _binding = null
42     }
43 }

```

## activity\_main.xml

Tabel 6. Source Layout XML activity\_main.xml

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <FrameLayout
3
4     xmlns:android="http://schemas.android.com/apk/res/android"
5     xmlns:tools="http://schemas.android.com/tools"
6     android:id="@+id/frame_container"
7     android:layout_width="match_parent"
8     android:layout_height="match_parent"
9     tools:context=".MainActivity">
10 </FrameLayout>

```

**fragment\_detail.xml**

Tabel 7. Source Layout XML fragment\_detail.xml

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <androidx.constraintlayout.widget.ConstraintLayout         |
|    | xmlns:android="http://schemas.android.com/apk/res/android" |
| 3  | xmlns:tools="http://schemas.android.com/tools"             |
| 4  | android:layout_width="match_parent"                        |
| 5  | android:layout_height="match_parent"                       |
| 6  | xmlns:app="http://schemas.android.com/apk/res-auto"        |
| 7  | tools:context=".DetailFragment"                            |
| 8  | android:layout_margin="10dp">                              |
| 9  |                                                            |
| 10 | <ImageView                                                 |
| 11 | android:id="@+id/img_poster"                               |
| 12 | android:layout_width="wrap_content"                        |
| 13 | android:layout_height="400dp"                              |
| 14 | android:layout_marginTop="12dp"                            |
| 15 | android:contentDescription="Poster"                        |
| 16 | tools:src="@drawable/nwh"                                  |
| 17 | app:layout_constraintTop_toTopOf="parent"                  |
| 18 | app:layout_constraintStart_toStartOf="parent"              |
| 19 | />                                                         |
| 20 |                                                            |
| 21 | <TextView                                                  |
| 22 | android:id="@+id/tv_item_title"                            |
| 23 | android:layout_width="350dp"                               |
| 24 | android:layout_height="wrap_content"                       |
| 25 | tools:text="Ini Judul"                                     |
| 26 | app:layout_constraintTop_toBottomOf="@id/img_poster"       |
| 27 | app:layout_constraintStart_toStartOf="parent"              |
| 28 | android:textSize="24sp"                                    |
| 29 | android:textStyle="bold"                                   |
| 30 | android:textColor="@color/black"                           |
| 31 | />                                                         |
| 32 |                                                            |
| 33 | <TextView                                                  |
| 34 | android:id="@+id/tv_item_years"                            |
| 35 | android:layout_width="wrap_content"                        |
| 36 | android:layout_height="wrap_content"                       |
| 37 | android:layout_marginTop="4dp"                             |
| 38 | tools:text="2020"                                          |
| 39 | android:textSize="20sp"                                    |
| 40 | app:layout_constraintEnd_toEndOf="parent"                  |
| 41 | app:layout_constraintHorizontal_bias="1.0"                 |
| 42 | app:layout_constraintStart_toEndOf="@id/tv_item_title"     |
| 43 | app:layout_constraintTop_toBottomOf="@id/img_poster"       |
|    | />                                                         |
| 44 |                                                            |
| 45 | <TextView                                                  |

```

46         android:id="@+id/tv_item_plot"
47         android:layout_width="wrap_content"
48         android:layout_height="wrap_content"
49         tools:text="Plot :"
50         app:layout_constraintStart_toStartOf="parent"
51
52     app:layout_constraintTop_toBottomOf="@id/tv_item_title"
53         android:textSize="16sp"
54         android:textStyle="bold"
55         android:textColor="@color/black"
56     />
57     <TextView
58         android:id="@+id/tv_item_desc"
59         android:layout_width="wrap_content"
60         android:layout_height="wrap_content"
61         tools:text="deskripsi"
62
63     app:layout_constraintTop_toBottomOf="@id/tv_item_plot"
64         app:layout_constraintStart_toStartOf="parent"
65         android:textSize="16sp"
66         android:textStyle="bold"
67         android:textColor="@color/black"
68     />
69 </androidx.constraintlayout.widget.ConstraintLayout>

```

## fragment\_home.xml

Tabel 8. Source Layout XML fragment\_home.xml

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      tools:context=".HomeFragment">
9
10     <androidx.recyclerview.widget.RecyclerView
11         android:id="@+id/rv_movie"
12         android:layout_width="0dp"
13         android:layout_height="0dp"
14         android:layout_margin="15dp"
15         app:layout_constraintBottom_toBottomOf="parent"
16         app:layout_constraintEnd_toEndOf="parent"
17         app:layout_constraintStart_toStartOf="parent"
18         app:layout_constraintTop_toTopOf="parent" />
19 </androidx.constraintlayout.widget.ConstraintLayout>

```

**item\_movie.xml**

Tabel 9. Source Layout XML item\_movie.xml

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <androidx.cardview.widget.CardView                         |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 4  | xmlns:card_view="http://schemas.android.com/apk/res-auto"  |
| 5  | xmlns:tools="http://schemas.android.com/tools"             |
| 6  | android:id="@+id/card_view"                                |
| 7  | android:layout_width="match_parent"                        |
| 8  | android:layout_height="wrap_content"                       |
| 9  | android:layout_gravity="center"                            |
| 10 | android:layout_marginStart="8dp"                           |
| 11 | android:layout_marginTop="4dp"                             |
| 12 | android:layout_marginEnd="8dp"                             |
| 13 | android:layout_marginBottom="4dp"                          |
| 14 | card_view:cardCornerRadius="4dp">                          |
| 15 | <androidx.constraintlayout.widget.ConstraintLayout         |
| 16 | android:layout_width="match_parent"                        |
| 17 | android:layout_height="wrap_content"                       |
| 18 | android:padding="8dp"                                      |
| 19 | android:background="@color/gray">                          |
| 20 |                                                            |
| 21 | <ImageView                                                 |
| 22 | android:id="@+id/img_poster"                               |
| 23 | android:layout_width="70dp"                                |
| 24 | android:layout_height="100dp"                              |
| 25 | android:scaleType="centerCrop"                             |
| 26 | card_view:layout_constraintBottom_toBottomOf="parent"      |
| 27 | card_view:layout_constraintStart_toStartOf="parent"        |
| 28 | card_view:layout_constraintTop_toTopOf="parent"            |
| 29 | tools:src="@drawable/nwh" />                               |
| 30 |                                                            |
| 31 | <TextView                                                  |
| 32 | android:id="@+id/tv_item_title"                            |
| 33 | android:layout_width="200dp"                               |
| 34 | android:layout_height="wrap_content"                       |
| 35 | android:textColor="@color/white"                           |
| 36 | android:layout_marginStart="12dp"                          |
| 37 | android:textSize="24sp"                                    |
| 38 | android:textStyle="bold"                                   |
| 39 | tools:layout_editor_absoluteX="78dp"                       |
| 40 | tools:layout_editor_absoluteY="2dp"                        |
| 41 | card_view:layout_constraintStart_toEndOf="@id/img_poster"  |

```

42     card_view:layout_constraintEnd_toStartOf="@id/tv_item_years"
43         tools:text="Ini Judul" />
44
45         <TextView
46             android:id="@+id/tv_item_years"
47             android:layout_width="wrap_content"
48             android:layout_height="wrap_content"
49             android:layout_marginStart="16dp"
50             android:layout_marginTop="12dp"
51             tools:text="2020"
52             android:textColor="@color/white"
53             card_view:layout_constraintTop_toTopOf="parent"
54
55     card_view:layout_constraintStart_toEndOf="@id/tv_item_title"
56         tools:layout_editor_absoluteY="29dp" />
57
58         <TextView
59             android:id="@+id/tv_item_plot"
60             android:layout_width="wrap_content"
61             android:layout_height="wrap_content"
62             android:layout_marginTop="8dp"
63             android:layout_marginStart="12dp"
64             tools:text="Plot :"
65             android:textColor="@color/white"
66             android:textSize="15sp"
67             android:textStyle="bold"
68
69     card_view:layout_constraintStart_toEndOf="@id/img_poster"
70
71     card_view:layout_constraintTop_toBottomOf="@id/tv_item_title"
72 />
73
74         <TextView
75             android:id="@+id/tv_item_desc"
76             android:layout_width="0dp"
77             android:layout_height="wrap_content"
78             android:layout_marginStart="12dp"
79             android:textColor="@color/white"
80
81     card_view:layout_constraintTop_toTopOf="@id/tv_item_plot"
82
83     card_view:layout_constraintStart_toEndOf="@id/tv_item_plot"
84         card_view:layout_constraintEnd_toEndOf="parent"
85         tools:layout_editor_absoluteY="42dp"
86         tools:text="Deskripsi Film" />
87
88         <Button
89             android:id="@+id/btn_detail"
90             android:layout_width="wrap_content"
91             android:layout_height="wrap_content"
92             android:layout_marginTop="4dp"
93             android:text="@string/btn_detail"

```



|     |                                                                                                  |
|-----|--------------------------------------------------------------------------------------------------|
| 88  | <code>card_view:layout_constraintEnd_toEndOf="parent"</code>                                     |
| 89  | <code>card_view:layout_constraintHorizontal_bias="0.508"</code>                                  |
| 90  | <code>card_view:layout_constraintStart_toEndOf="@id/btn_imdb"</code>                             |
| 91  | <code>card_view:layout_constraintTop_toBottomOf="@+id/tv_item_desc"</code><br><code>/&gt;</code> |
| 92  |                                                                                                  |
| 93  | <code>&lt;Button</code>                                                                          |
| 94  | <code>android:id="@+id/btn_imdb"</code>                                                          |
| 95  | <code>android:layout_width="wrap_content"</code>                                                 |
| 96  | <code>android:layout_height="wrap_content"</code>                                                |
| 97  | <code>android:layout_marginStart="144dp"</code>                                                  |
| 98  | <code>android:layout_marginTop="4dp"</code>                                                      |
| 99  | <code>android:text="@string/btn_imdb"</code>                                                     |
| 100 | <code>card_view:layout_constraintStart_toStartOf="parent"</code>                                 |
| 101 | <code>card_view:layout_constraintTop_toBottomOf="@id/tv_item_desc"</code><br><code>/&gt;</code>  |
| 102 |                                                                                                  |
| 103 | <code>&lt;/androidx.constraintlayout.widget.ConstraintLayout&gt;</code>                          |
| 104 | <code>&lt;/androidx.cardview.widget.CardView&gt;</code>                                          |

## Compose:

### string.xml

Tabel 10. Source String Data Jetpack Compose

|                                                                                    |
|------------------------------------------------------------------------------------|
| <code>&lt;resources&gt;</code>                                                     |
| <code>&lt;string name="app_name"&gt;Movie List&lt;/string&gt;</code>               |
| <code>&lt;string name="title1"&gt;Iron Man 3&lt;/string&gt;</code>                 |
| <code>&lt;string name="title2"&gt;Captain America: Civil War&lt;/string&gt;</code> |
| <code>&lt;string name="title3"&gt;Avengers: Endgame&lt;/string&gt;</code>          |
| <code>&lt;string name="title4"&gt;Avengers: Infinity War&lt;/string&gt;</code>     |
| <code>&lt;string name="title5"&gt;Spider-man: No Way Home&lt;/string&gt;</code>    |
| <code>&lt;string</code>                                                            |
| <code>name="url1"&gt;https://www.imdb.com/title/tt1300854/?ref_=nv_sr_sr</code>    |
| <code>sg_0_tt_8_nm_0_in_0_q_Iron%2520Man%25203&lt;/string&gt;</code>               |
| <code>&lt;string</code>                                                            |
| <code>name="url2"&gt;https://www.imdb.com/title/tt3498820/?ref_=nv_sr_sr</code>    |
| <code>sg_0_tt_6_nm_0_in_0_q_captain%2520civil%2520war&lt;/string&gt;</code>        |
| <code>&lt;string</code>                                                            |
| <code>name="url3"&gt;https://www.imdb.com/title/tt4154756/?ref_=nv_sr_sr</code>    |
| <code>sg_0_tt_8_nm_0_in_0_q_infinity%2520war&lt;/string&gt;</code>                 |
| <code>&lt;string</code>                                                            |
| <code>name="url4"&gt;https://www.imdb.com/title/tt4154796/?ref_=nv_sr_sr</code>    |

```

sg_0_tt_8_nm_0_in_0_q_endgame</string>
  <string
name="url5">https://www.imdb.com/title/tt10872600/?ref_=nv_sr_s
rsg_0_tt_8_nm_0_in_0_q_no%2520way</string>

    <string name="releaseDate1">2013</string>
    <string name="releaseDate2">2016</string>
    <string name="releaseDate3">2019</string>
    <string name="releaseDate4">2018</string>
    <string name="releaseDate5">2021</string>

    <string name="desc1">Brilliant industrialist Tony Stark /
Iron Man faces an enemy that knows no bounds. Tony world is
destroyed and he investigates who is responsible for the
destruction. Armed with his tools and instincts, Tony sets out
to save those he loves. An old question comes back to haunt
him; did humans make the Iron Man suit or did the suit define
his identity as a human?</string>
    <string name="desc2">The heavy civilian casualties have led
the government to control the superheroes. This heated
situation has Iron Man and Captain America camps opposing each
other. The battle is inevitable.</string>
    <string name="desc3">The remaining Avengers try to reverse
Thanos actions that destroyed half the universe. They travel
through time to collect the Infinity Stones, then use the
energy of the stones to go back in time and change the events
that caused the Snap.</string>
    <string name="desc4">The Avengers and their allies must be
willing to sacrifice everything in an attempt to defeat the
powerful Thanos before he succeeds in destroying the
universe.</string>
    <string name="desc5">For the first time in Spider-Man big-
screen history, the affable hero true identity is revealed,
causing his superpower responsibilities to clash with his
normal life, and putting everyone closest to him at
risk.</string>

    <string name="description1">Brilliant industrialist Tony
Stark / Iron Man faces an enemy that knows no bounds. Tony
world is destroyed and he investigates who is responsible for
the destruction. Armed with his tools and instincts, Tony sets
out to save those he loves. An old question comes back to haunt
him; did humans make the Iron Man suit or did the suit define
his identity as a human?</string>
    <string name="description2">The heavy civilian casualties
have led the government to control the superheroes. This heated
situation has Iron Man and Captain America camps opposing each
other. The battle is inevitable.</string>
    <string name="description3">The remaining Avengers try to
reverse Thanos actions that destroyed half the universe. They
travel through time to collect the Infinity Stones, then use
the energy of the stones to go back in time and change the

```

```

events that caused the Snap.</string>
    <string name="description4">The Avengers and their allies
must be willing to sacrifice everything in an attempt to defeat
the powerful Thanos before he succeeds in destroying the
universe.</string>
    <string name="description5">For the first time in Spider-
Man big-screen history, the affable hero true identity is
revealed, causing his superpower responsibilities to clash with
his normal life, and putting everyone closest to him at
risk.</string>
</resources>

```

## DataMovie.kt

Tabel 11. Source Data Class Movie Jetpack Compose

```

1 package com.example.mlistcompose.data
2
3 import com.example.mlistcompose.R
4 import com.example.mlistcompose.model.ListMovie
5
6 class DataMovie{
7     fun loadMovieList(): List<ListMovie> = listOf(
8         ListMovie(R.string.title1, R.string.url1,
9         R.drawable.ironman3, R.string.releaseDate1, R.string.desc1,
10        R.string.description1),
11        ListMovie(R.string.title2, R.string.url2,
12        R.drawable.civilwar, R.string.releaseDate2, R.string.desc2,
13        R.string.description2),
14        ListMovie(R.string.title3, R.string.url3,
15        R.drawable.infinitywar, R.string.releaseDate3,
16        R.string.desc3, R.string.description3),
17        ListMovie(R.string.title4, R.string.url4,
18        R.drawable.endgame, R.string.releaseDate4, R.string.desc4,
19        R.string.description4),
20        ListMovie(R.string.title5, R.string.url5,
21        R.drawable.nwh, R.string.releaseDate5, R.string.desc5,
22        R.string.description5)
23    )
24 }

```

## ListMovie.kt

Tabel 12. Source List Movie Jetpack Compose

```

1 package com.example.mlistcompose.model
2
3 import androidx.annotation.DrawableRes
4 import androidx.annotation.StringRes
5
6 data class ListMovie(

```

```

7      @StringRes val title: Int,
8      @StringRes val url: Int,
9      @DrawableRes val image: Int,
10     @StringRes val years: Int,
11     @StringRes val desc: Int,
12     @StringRes val description: Int
13 )

```

## DetailActivity.kt

Tabel 13. Source Detail Activity Jetpack Compose

```

1  package com.example.mlistcompose
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.compose.foundation.Image
7  import androidx.compose.material3.Surface
8  import androidx.compose.material3.Text
9  import androidx.compose.ui.Alignment
10 import androidx.compose.ui.Modifier
11 import androidx.compose.foundation.layout.*
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.res.painterResource
14 import androidx.compose.ui.res.stringResource
15 import androidx.compose.ui.text.TextStyle
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import androidx.compose.ui.unit.sp
19 import com.example.mlistcompose.ui.theme.MListComposeTheme
20
21 class DetailActivity : ComponentActivity() {
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24         val title = intent.getIntExtra("title", 0)
25         val years = intent.getIntExtra("years", 0)
26         val image = intent.getIntExtra("image", 0)
27         val description = intent.getIntExtra("description",
28         0)
29
30         setContent {
31             MListComposeTheme {
32                 Surface(
33                     modifier = Modifier.fillMaxSize(),
34                     color = Color.Black
35                 ) {
36                     Column (
37                         modifier = Modifier
38                             .fillMaxWidth()
39                             .height(200.dp)

```

```

39         .padding(16.dp)
40     ){
41         Image(
42             modifier =
43             Modifier.height(225.dp).fillMaxWidth(),
44             painter = painterResource(image),
45             contentDescription =
46             stringResource(title)
47         )
48         Row (
49             modifier = Modifier
50             .fillMaxWidth()
51         ){
52             Text(
53                 text = stringResource(title),
54                 style = TextStyle(
55                     fontSize = 40.sp,
56                     fontWeight =
57                     FontWeight.Bold,
58                     color = Color.White
59                 ),
60                 modifier =
61                 Modifier.padding(bottom = 4.dp)
62                 .weight(1F)
63             )
64             Text(
65                 text = stringResource(years),
66                 style = TextStyle(
67                     fontSize = 24.sp,
68                     color = Color.White
69                 ),
70                 modifier =
71                 Modifier.padding(bottom = 4.dp)
72             )
73             .wrapContentWidth(Alignment.End)
74             .align(Alignment.CenterVertically)
75         )
76     }
77     Text(
78         text = "Plot :",
79         style = TextStyle(
80             fontSize = 16.sp,
81             fontWeight =
82             FontWeight.Bold,
83             color = Color.White
84         )
85     )
86     Text(
87         text =
88         stringResource(description),
89         style = TextStyle(

```

|    |                     |
|----|---------------------|
| 82 | fontSize = 16.sp,   |
| 83 | color = Color.White |
| 84 | )                   |
| 85 | )                   |
| 86 | }                   |
| 87 | }                   |
| 88 | }                   |
| 89 | }                   |
| 90 | }                   |
| 91 | }                   |

## MainActivity.kt

Tabel 14. Source Main Activity Jetpack Compose

|    |                                                             |
|----|-------------------------------------------------------------|
| 1  | package com.example.mlistcompose                            |
| 2  |                                                             |
| 3  | import android.content.Intent                               |
| 4  | import android.os.Bundle                                    |
| 5  | import androidx.activity.ComponentActivity                  |
| 6  | import androidx.activity.compose.setContent                 |
| 7  | import androidx.activity.enableEdgeToEdge                   |
| 8  | import androidx.compose.foundation.Image                    |
| 9  | import androidx.compose.foundation.layout.Arrangement       |
| 10 | import androidx.compose.foundation.layout.Column            |
| 11 | import androidx.compose.foundation.layout.PaddingValues     |
| 12 | import androidx.compose.foundation.layout.Spacer            |
| 13 | import androidx.compose.foundation.layout.fillMaxWidth      |
| 14 | import androidx.compose.foundation.layout.padding           |
| 15 | import androidx.compose.foundation.layout.wrapContentHeight |
| 16 | import androidx.compose.foundation.lazy.LazyColumn          |
| 17 | import androidx.compose.foundation.shape.RoundedCornerShape |
| 18 | import androidx.compose.material3.Card                      |
| 19 | import androidx.compose.material3.CardDefaults              |
| 20 | import androidx.compose.runtime.Composable                  |
| 21 | import androidx.compose.ui.Modifier                         |
| 22 | import androidx.compose.ui.draw.clip                        |
| 23 | import androidx.compose.material3.Text                      |
| 24 | import androidx.compose.ui.layout.ContentScale              |
| 25 | import androidx.compose.ui.unit.dp                          |
| 26 | import androidx.compose.foundation.lazy.items               |
| 27 | import androidx.compose.ui.platform.LocalContext            |
| 28 | import androidx.compose.ui.res.painterResource              |
| 29 | import androidx.compose.foundation.layout.Row               |
| 30 | import androidx.compose.foundation.layout.fillMaxSize       |
| 31 | import androidx.compose.foundation.layout.size              |
| 32 | import androidx.compose.foundation.layout.width             |
| 33 | import androidx.compose.foundation.layout.wrapContentWidth  |
| 34 | import androidx.compose.material3.Button                    |
| 35 | import androidx.compose.ui.res.stringResource               |
| 36 | import androidx.compose.ui.tooling.preview.Preview          |

```

37 import com.example.mlistcompose.model.ListMovie
38 import com.example.mlistcompose.data.DataMovie
39 import androidx.compose.material3.MaterialTheme
40 import androidx.compose.material3.Surface
41 import androidx.compose.ui.Alignment
42 import androidx.core.net.toUri
43 import androidx.compose.ui.text.TextStyle
44 import androidx.compose.ui.text.font.FontWeight
45 import androidx.compose.ui.unit.sp
46 import androidx.compose.ui.graphics.Color
47 import com.example.mlistcompose.ui.theme.MListComposeTheme
48
49 class MainActivity : ComponentActivity() {
50     override fun onCreate(savedInstanceState: Bundle?) {
51         super.onCreate(savedInstanceState)
52         enableEdgeToEdge()
53         setContent {
54             MListComposeTheme {
55                 Surface (
56                     modifier = Modifier.fillMaxSize(),
57                     color = Color.Black
58                 ){
59                     MovieListScreen(movie =
DataMovie().loadMovieList())
60                 }
61             }
62         }
63     }
64 }
65 @Composable
66 fun MovieListScreen(movie: List<ListMovie>) {
67     LazyColumn (contentPadding = PaddingValues(16.dp),
verticalArrangement = Arrangement.spacedBy(16.dp))
68     {
69         items(movie){item ->
70             MovieItems(movie = item)
71         }
72     }
73 }
74 @Composable
75 fun MovieItems(movie: ListMovie){
76     val context = LocalContext.current
77     val url = stringResource(movie.url)
78     val releaseDate = stringResource(movie.years)
79
80     Card(
81         shape = RoundedCornerShape(16.dp),
82         modifier = Modifier
83             .fillMaxWidth()
84             .wrapContentHeight(),
85         colors = CardDefaults.cardColors(
86             containerColor = Color(0xFF343539)

```

```

87         ),
88         elevation = CardDefaults.cardElevation(4.dp)
89     ) {
90         Row(
91             modifier = Modifier
92                 .padding(16.dp)
93                 .fillMaxWidth()
94         ) {
95             Image(
96                 painter = painterResource(movie.image),
97                 contentDescription =
stringResource(movie.title),
98                 contentScale = ContentScale.Crop,
99                 modifier = Modifier
100                     .size(width = 90.dp, height = 200.dp)
101                     .clip(RoundedCornerShape(8.dp))
102                     .align(Alignment.CenterVertically)
103             )
104             Spacer(modifier = Modifier.width(12.dp))
105             Column(
106                 modifier = Modifier
107                     .fillMaxWidth()
108                     .align(Alignment.CenterVertically)
109             ) {
110                 Row (
111                     modifier = Modifier
112                         .fillMaxWidth()
113                         .padding(8.dp)
114                 ){
115                     Text(
116                         text = stringResource(movie.title),
117                         style = TextStyle(
118                             fontSize = 26.sp,
119                             fontWeight = FontWeight.Bold,
120                             color = Color.White
121                         ),
122                     modifier = Modifier.padding(bottom =
4.dp)
123                         .weight(1F)
124                     )
125                     Text(
126                         text = releaseDate,
127                         style = TextStyle(
128                             fontSize = 20.sp,
129                             color = Color.White
130                         ),
131                     modifier = Modifier.padding(bottom =
4.dp)
132                         .wrapContentWidth(Alignment.End)
133                 ).align(Alignment.CenterVertically)
134             )

```



```

135         }
136         Row (
137             modifier = Modifier
138                 .fillMaxWidth()
139         ){
140             Text(
141                 text = "Plot :",
142                 style = TextStyle(
143                     fontSize = 16.sp,
144                     fontWeight = FontWeight.Bold,
145                     color = Color.White
146                 )
147             )
148             Text(
149                 text = stringResource(movie.desc),
150                 style = TextStyle(
151                     fontSize = 16.sp,
152                     color = Color.White
153                 ),
154                 modifier = Modifier.padding(bottom =
155                 8.dp)
156             )
157         }
158         Row(
159             modifier = Modifier.fillMaxWidth(),
160             horizontalArrangement = Arrangement.End
161         ) {
162             Button(onClick = {
163                 val intent =
164                 Intent(Intent.ACTION_VIEW, url.toUri())
165                 context.startActivity(intent)
166             }) {
167                 Text("IMDB")
168             }
169             Spacer(modifier = Modifier.width(4.dp))
170             Button(onClick = {
171                 val intent = Intent(context,
172                 DetailActivity::class.java).apply {
173                     putExtra("title", movie.title)
174                     putExtra("years", movie.years)
175                     putExtra("image", movie.image)
176                     putExtra("description",
177                     movie.description)
178                 }
179                 context.startActivity(intent)
180             }) {
181                 Text("Detail")
182             }
183         }
184     }
185 }

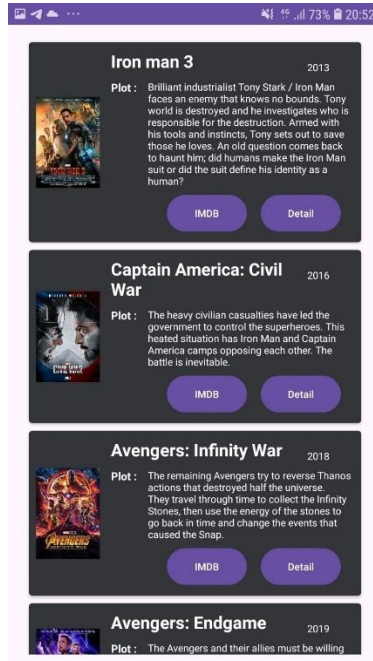
```

```

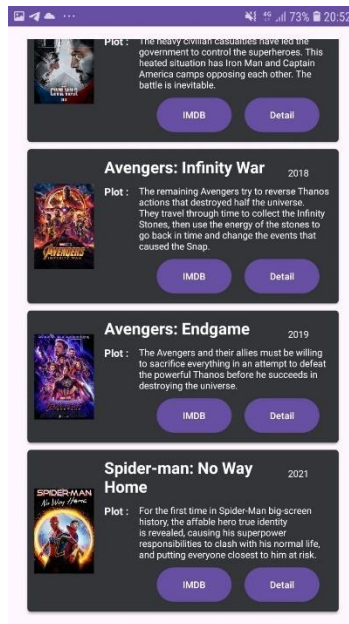
183     }
184 }
185 @Preview(showBackground = true)
186 @Composable
187 fun GreetingPreview() {
188     MListComposeTheme {
189         MovieListScreen(movie = DataMovie().loadMovieList())
190     }
191 }

```

## B. Output Program



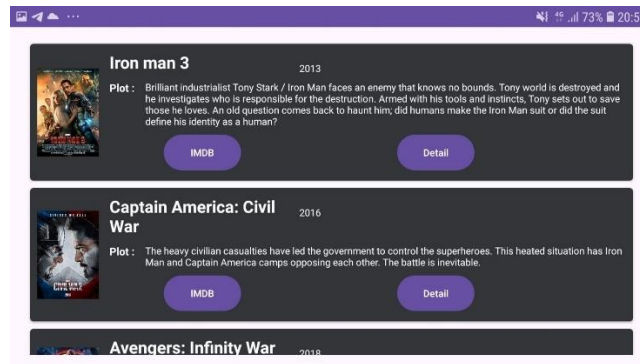
Gambar 3. Tampilan Awal UI XML



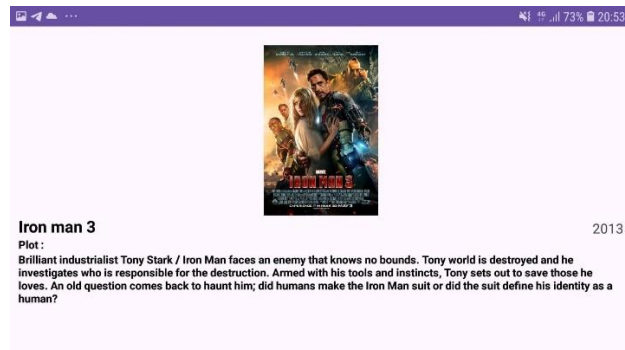
Gambar 4. Tampilan Ketika di Scroll XML



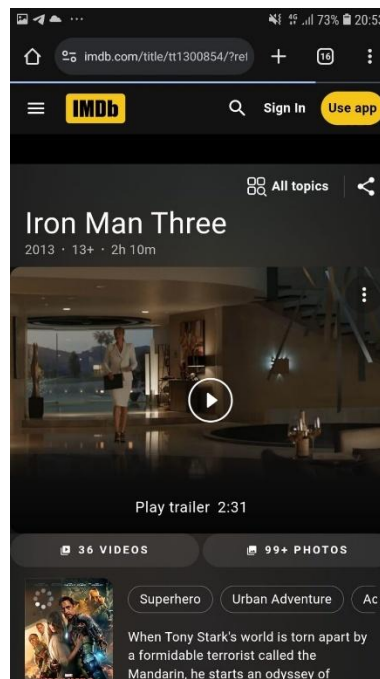
Gambar 5. Tampilan Tombol Detail Ditekan XML



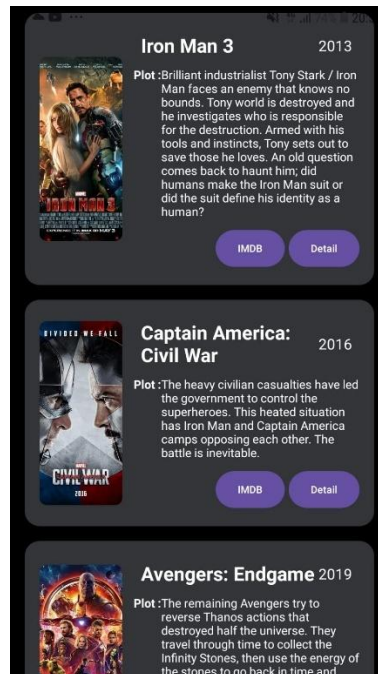
Gambar 6. Tampilan Awal UI di Rotate XML



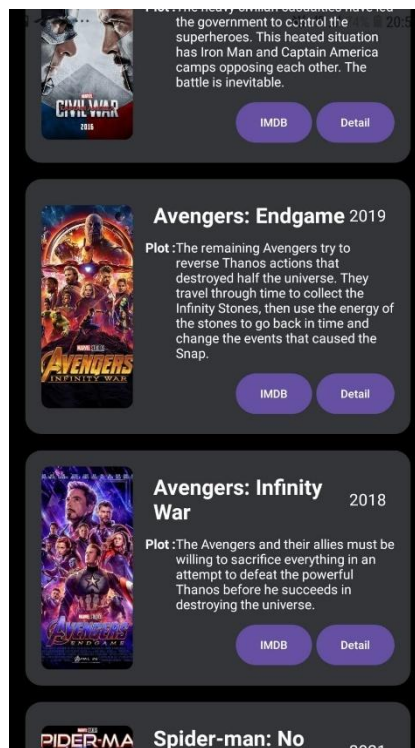
Gambar 7. Tampilan Detail UI di Rotate XML



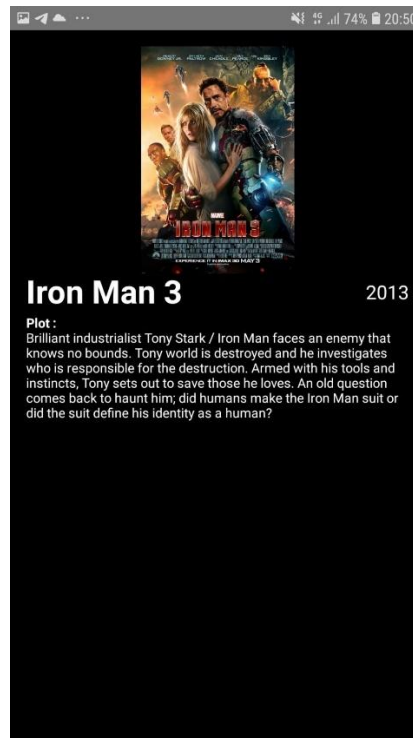
Gambar 8. Tombol IMDB di Tekan



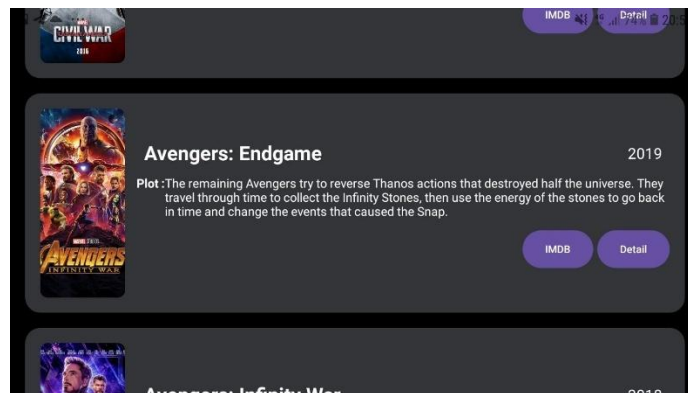
Gambar 9. Tampilan Awal UI Jetpack Compose



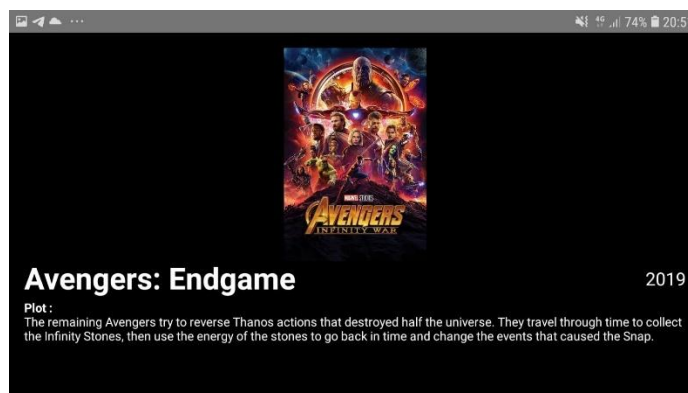
Gambar 10. Tampilan UI Ketika di Scroll Jetpack Compose



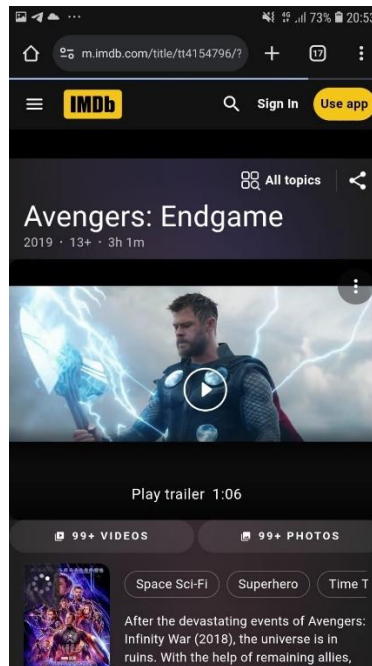
Gambar 11. Tampilan Detail UI Jetpack Compose



Gambar 12. Tampilan Awal UI di Rotate Jetpack Compose



Gambar 13. Tampilan Detail UI di Rotate Jetpack Compose



Gambar 14. Tampilan Ketika Tombol IMDB di Tekan Jetpack Compose

## C. Pembahasan

### XML

#### MainActivity.kt

Pada baris [1 – 5], mendeklarasikan package dan import yang digunakan untuk definisi file merupakan bagian dari package `com.example.movielistxml`

Pada baris [7], mendeklarasikan activity utama dari `AppCompatActivity`, yang merupakan `MainActivity` adalah layar utama aplikasi

Pada baris [8 – 10], fungsi lifecycle dipanggil saat activity dibuat pertama kali dan memastikan bahwa implementasi `onCreate` dari superclass tetap jalan. Selain itu mengatur layout xml akan digunakan sebagai UI untuk activity ini.

Pada baris [12 – 13], mendapatkan `FragmentManager` untuk mengelola activity dan membuat instance dari `HomeFragment`

Pada baris [14 – 15], mengecek apakah Fragment dengan tag nama kelas `HomeFragment` sudah pernah ditambah, jika belum maka blok if akan jalan.

Pada baris [16 – 21], mencetak log debugging bahwa fragment sudah ditambah, memulai proses perubahan, menambahkan `homeFragment` ke dalam container `R.id.frame_container` dan commit untuk menyimpan dan menjalankan perubahan transaksi fragment

#### Movie.kt

Pada baris [1 – 4]. Mendeklarasikan file dalam package yang sama dan interface untuk memungkinkan objek dikirim antar komponen android antar activity atau fragment. Parcelize untuk anotasi dari Kotlin extension untuk menghindari penulisan parcelable secara manual.

Pada baris [6 – 14], menginstruksikan Kotlin secara otomatis menghasilkan parcelable untuk kelas ini dan data yang digunakan untuk menyimpan dan mengelola data secara sederhana. Terdapat beberapa properti seperti title, url, image, plot, years, dan desc. Parcelable menandakan bahwa objek movie bisa diparcel dan dikirim dengan intent atau bundle.

### **ListAdapterMovie.kt**

Pada baris [1 – 9], mengimpor komponen android yang dibutuhkan untuk membuat dan mengatur tampilan daftar menggunakan RecyclerView

Pada baris [12 – 16], berisi class yang berisi listMovie sebagai daftar film yang akan ditampilkan, lambda function yang menerima url dan detail film saat di tekan. Selain itu mewarisi RecyclerView.Adapter dan ListViewAdapter sebagai ViewHoldernya

Pada baris [18-26], ViewHolder ini menyimpan referensi ke semua komponen UI dari item layout item\_movie.xml

Pada baris [27 – 30], membuat objek ViewHolder baru dnegan menginflate layout item\_movie.xml dan mengembalikannya.

Pada baris [31], RecyclerView bisa tahu berapa banyak item harus ditampilkan sesuai data yang ada

Pada baris [33 – 42], mengisi data ke tampilan berdasarkan posisi item di daftar dan juga mengatur listener pada tombol IMDB dan Detail yang memanggil fungsi onImdbClick dan onDetailClick dengan data lengkap film tersebut

### **HomeFragment.kt**

Pada baris [1 – 11], mengimport komponen UI, intent dan binding layout dengan ViewBinding FragmentHomeBinding untuk akses view dari xml

Pada baris [13 - 16], mendeklarasikan class dan variable binding, \_binding untuk menyimpan hasil inflasi dari FragmentHomeBindng dan binding non null untuk akses elemen UI

Pada baris [18 – 19]. Mendeklarsikan adapter dan data list untuk adapter RecyclerView dan list untuk menyimpan daftar film dari resource

Pada baris [21 – 32], inflate layout fragment menggunakan viewbinding, mengisi list dengan data dari getListMovie dan memanggil setupRecyclerView untuk mengatur tampilan daftar film

Pada baris [33 – 62], Membuat adapter dengan dua aksi yang berupa imdb click dan detail click yang mengoper data film melalui bundle, selain itu mengatur RecyclerView dengan LinearLayoutManager (daftar vertikal)



Pada baris [64 – 78], mengambil data dari resource (string.xml) untuk membuat objek Movie dan dataImage.recycle untuk melepaskan resource setelah digunakan.

Pada baris [80 – 83], membersihkan binding saat view dihancurkan untuk mencegah memory leak

### **DetailFragment.kt**

Pada baris [1 – 8], mendeklarasi import untuk menampilkan UI detail dan mengimpor ViewBinding agar lebih aman dan ringkas saat mengakses view

Pada baris [11 – 14], mendeklarsi fragment dan binding untuk properti nullable yang diisi saat onCreateView dan binding untuk mengakses elemen UI tanpa nullable

Pada baris [16 – 20], menginflate layout XML fragment\_detail.xml dengan View Binding

Pada baris [22 – 26], mengambil data yang dikirim dari HomeFragment melalui Bundle

Pada baris [28 – 34], mengisi elemen-elemen tampilan dengan data yang diterima dan gambar hanya diatur jika image tidak null.

Pada baris [38 – 41], membersihkan binding saat view dihancurkan untuk mencegah memory leak

### **\*activity\_main.xml**

Pada baris [1 – 9], membuat ID di Kotlin MainActivity sebagai tempat menampung fragment, match parent akan mengisi seluruh layar dan contextnya akan terkait dengan MainActivity

Terdapat 2 activity\_activity main dengan isi yang sama tapi salah satu nya itu berfungsi sebagai landscape saat layar di rotate

### **\*fragment\_detail.xml**

Pada baris [10 – 19], <ImageView layout ini akan digunakan untuk penempatan gambar khusus poster film dengan id img\_poster yang digunakan pada kode Kotlin. Peletakkannya di bagian atas layout dan rata kiri

Pada baris [21 – 31], <TextView layout ini digunakan untuk penempatan title dengan id tv\_item\_title yang berada tepat di bawah poster.

Pada baris [33 – 43], <TextView layout ini digunakan untuk penempatan tahun dengan id tv\_item\_years yang terletak di sebelah kanan judul

Pada baris {45 – 55}, <TextView layout ini digunakan untuk penempatan label teks sebelum deskripsi plot dengan id tv\_item\_plot

Pada baris [57 – 67], <TextView layout ini digunakan untuk penempatan deskripsi film dengan id tv\_item\_desc

Terdapat 2 fragment\_detail juga dengan isi yang sama tapi yang satu berfungsi untuk menjaga tampilan saat landscape atau layar di rotate

**\*fragment\_home.xml**

Pada baris [1 – 19], membuat container fleksibel untuk penempatan berdasarkan constraint view dan ke parent, id rv\_movie diakses di HomeFragment untuk menampilkan list movie dengan beberapa aturan lebar, tinggi, margin dan layout constraint untuk RecyclerView memenuhi semua area parent dengan margin

**\*item\_movie.xml**

Pada baris [1 – 19], membuat constraint layout CardView untuk mengatur posisi elemen secara fleksible dan memberi warna latar abu abu

Pada baris [21 – 29], <ImageView untuk membuat layout gambar dengan scaletype center agar gambar memenuhi area dan constraint ke atas, bawah, kiri parent

Pada baris [31 – 43], <TextView untuk membuat judul film meletakkannya disebelah kanan poster dan diapit img poster dan tv\_item\_years

Pada baris [45 – 55], <TextView untuk membuat tahun film yang peletakkannya di kanan judul dan menampilkan rilis filmnya kapan.

Pada baris [57 – 68], <TextView untuk membuat label plot di bawah judul dan dikanan poster

Pada baris [70 – 80], <TextView untuk membuat deskripsi film di sebelah label plot dan menampilkan ringkasan cerita film

Pada baris [82 – 91], <Button untuk tombol detail pindah ke DetailFragment di bawah deskripsi, dikanan bersebelahan dengan tombol IMDB

Pada baris [93 – 101], <Button untuk tombol pindah halaman ke link IMDB dan terletak di bawah deskripsi di kiri.

**Jetpack Compose****\*strings.xml**

Pada string ini, berisikan data data yang akan digunakan pada kode Kotlin yang akan dijelaskan setelahnya. StringArray ini berfungsi sebagai data yang akan digunakan untuk menampilkan film atau deskripsi dan semacamnya.

**DataMovie.kt**

Pada baris [1 – 4], menempatkan file di package data, mengimpor R untuk akses ke resource seperti R.string atau R.drawable. mengimpor ListMovie dari class model yang merepresentasikan satu film

Pada baris [6 -13], membuat fungsi loadMovieList untuk menghasilkan daftar film dari List<ListMovie>, mengembalikan data statis yang isinya objek ListMovie dan data ini akan digunakan untuk ditampilkan di UI di LazyColumn, listOf mempunyai format yang nilainya akan diambil dari strings.xml atau drawable

### **ListMovie.kt**

Pada baris [1 – 4], menempatkan file di package model karena representasi data, @StringRes dan @DrawableRes untuk anotasi validasi tipe resource

Pada baris [6 – 13], membuat data class untuk menyimpan data film yang masing masing bertipe data Int karena akan mengambil dari resource ID.

### **DetailActivity.kt**

Pada baris [1 – 19], membuat activity dasar compose dan mengimpor lainnya seperti elemen UI compose seperti Image, Text, Column, Row dan sebagainya

Pada baris [21 – 22], membuat activity yang digunakan untuk menampilkan detail film yang dipilih dari list, dan onCreate untuk format awal eksekusi UI

Pada baris [24 – 27], mengambil data dari intent dan data dikirim dalam bentuk resource ID sesuai @StringRes dan @DrawableRes

Pada baris [29 – 34], surface untuk memberi tempat dengan latar hitam dan MListComposeTheme membungkus UI agar mengikuti tema aplikasi

Pada baris [35 – 40], menggunakan column untuk menampilkan elemen secara vertikal

Pada baris [41 – 45], menampilkan gambar dari resource ID dan contentDescription diisi dengan judul film untuk aksesibilitas

Pada baris [46 – 70], menggunakan row agar judul dan tahun sejajar secara horizontal dan weight 1F pada judul agar mengisi ruang, dan tahun mengikuti di kanan

Pada baris [71 – 85], Text yang akan digunakan untuk label dan isi deskripsi film

### **MainActivity.kt**

Pada baris [1 – 46], package yang menandakan bahwa file ini berada di package yang sama dan mengimport elemen elemen dari jetpack compose dan tema dari aplikasi MListComposeTheme

Pada baris [48 – 63], activity utama yang memanggil MovieListScreen() saat app dibuka dengan menggunakan tema compose dan meletakkan di surface untuk interface dengan background hitam. Data list film diambil dari DataMovie().loadMovieList()

Pada baris [64 – 72], LazyColumn membuat list scrollable dan items(movie) menampilkan setiap elemen dengan MovieItems()

Pada baris [73 – 77], mengambil context dari compose untuk keperluan Intent dan mengambil resource string dari Int resource ID (url, years)

Pada baris [79 – 183], membuat card berisi satu item film dengan sudut membulat dan warna abu abu gelap. Membuat baris yang digunakan untuk menata elemen secara horizontal dan colum yang digunakan untuk Menyusun elemen secara vertikal. Image untuk menampilkan

gambar film dengan ukuran 90x200dp dan menggunakan gambar yang diberikan di movie.image. Spacer untuk memberi jarak antara elemen baris sebanyak 12dp. Text menampilkan teks yang ada pada elemen seperti judul film, tahun rilis, dan deskripsi plot dan disesuaikan textstyle seperti ukuran font dan ketebalan huruf. Fungsi button onClick untuk menampilkan tombol ketika membuka link IMDB dan tombol detail layar film. Selain itu ketika IMBD ditekan akan intent ke browser yang mengarahkan ke URL IMDB, detail akan diarahkan ke DetailActivity yang akan menampilkan informasi lebih detail mengenai film

Pada baris [184 – 190], membuat preview dari composable MovieListScreen untuk melihat tampilan aplikasi ketika dijalankan, selain itu juga memanggil MovieListScreen dengan data film yang dihasilkan oleh DataMovie().loadMovieList()

#### **D. Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Akmlsybn/Praktikum-Mobile>

## SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

### A. Pembahasan

RecyclerView sendiri masih digunakan karena salah satu komponen yang kuat dan serbaguna untuk menanggapi kumpulan data besar dengan mendaur ulang dan menggunakan kembali. Komponen ini menawarkan fleksibilitas dalam tata letak item dan mendukung fitur-fitur seperti pengguliran halus, animasi item, dan penampung tampilan. RecyclerView juga telah banyak digunakan untuk daftar yang kompleks dan dinamis. Meskipun kekurangannya boiler-plate tapi akan sangat mudah untuk dikuasai meskipun itu boilerplate.

Dibandingkan dengan LazyColumn ini karena relative baru di jetpack maka harus membiasakan dengan kerangka kerja compose. Meskipun singkat tapi butuh waktu untuk memahami LazyColumn ini. Pada hal ini orang lebih memilih boilerplate tapi dinamis dan kompleks dibandingkan harus beradaptasi dengan LazyColumn yang tergolong baru. Tapi hal ini bukan berarti buruk bagi LazyColumn, hanya saja butuh waktu lagi untuk mempelajari hal baru.